Chapter 4 — Strings

Text is everywhere — user names, file paths, log lines, JSON, HTML, SQL, chat messages — and in Python it all lives in one type, `str`. This chapter covers what intermediate developers must know cold: immutability, slicing, the formatting mini-language, the useful methods and their costs, efficient joining, Unicode vs bytes (UTF-8), and a working knowledge of regular expressions.

What you'll learn

1. Creating, indexing and looping over strings.
2. Immutability and how to "change" a string anyway.
3. Operators (`+`, `*`, `in`, comparisons, `%`) and slicing.
4. Escape sequences and raw strings.
5. f-strings, the format-spec mini-language, `str.format()`, `%`.
6. The string methods you'll actually use, with costs.
7. Joining many pieces efficiently.
8. Unicode vs bytes, encodings, UTF-8.
9. Regular expressions: `search`, `match`, `findall`, `sub`, groups.

Runnable code is in `code/`; every output was produced with Python 3.14 (code also runs on 3.12+).

4.1 What a string is — creating, indexing, looping

What it is

A **string** (`str`) is an ordered, immutable sequence of Unicode characters (Unicode gives every character in every writing system a number — §4.9). *Ordered*: each character has a position. *Immutable*: characters can never be changed. *Sequence*: supports `len()`, indexing, slicing, `in`, `for` — like lists and tuples.

There is no separate character type: a character is a `str` of length 1 (`type("x")` is `str`, `len("x")` is `1`).

Why it matters

Almost every program reads, cleans or produces text; exact indexing rules prevent off-by-one bugs.

How it works

1. **Create** with any of three quote styles (all give `str`):
   - `'single'` — convenient when the text contains `"`.
   - `"double"` — convenient with apostrophes: `"It's raining"`.
   - `"""triple"""` / `'''triple'''` — spans lines, holds both quote kinds; used for docstrings.
2. **Index** with `s[i]` from `0`. **Negative indexes** count from the right (`-1` is last). Valid range for length *n*: `0 … n−1` and `−n … −1`; anything else raises `IndexError`.
3. **Loop** with `for ch in s:`; `enumerate(s)` adds positions.

```
index:   0   1   2   3   4   5   6   7   8   9  10  11  12  13  14
char :   L   e   a   r   n   i   n   g   ␣   P   y   t   h   o   n
neg  : -15 -14 -13 -12 -11 -10  -9  -8  -7  -6  -5  -4  -3  -2  -1
```

```python
# code/01_string_basics.py (excerpt)
single = 'Hi there'
double = "It's raining"                      # apostrophe inside double quotes
triple = """Line one
Line two with "quotes" and 'apostrophes'"""
print(single, "|", double)
print(triple)
print(type("x"), len("x"))                   # a single character is just a str of length 1

word = "Learning Python"
print(word[0], word[9], word[-1], word[-6])  # L P n P
try:
    word[100]
except IndexError as e:
    print("IndexError:", e)

for i, ch in enumerate("abc"):
    print(f"{i}:{ch}", end=" ")
```

Output:

```
Hi there | It's raining
Line one
Line two with "quotes" and 'apostrophes'
<class 'str'> 1
L P n P
IndexError: string index out of range
0:a 1:b 2:c
```

Common mistakes

- Indexing starts at 0: the 8th character is `s[7]`.
- `for i in range(len(s)): s[i]` when you only need characters — write `for ch in s`.
- `'It's'` ends at the apostrophe → `SyntaxError`. Use `"It's"` or `'It\'s'`.

Self-check

1. `"Python programming is fun!"[-4]`? → `'f'`.

TL;DR

`str` = immutable, ordered sequence of Unicode characters; three quote styles; zero-based and negative indexes; `for ch in s`.

4.2 Immutability — and how to "update" a string

What it is

Immutable: the value can't change after creation. `s[0] = "J"` is illegal; `upper()`, `replace()`, `+` all return a **new** string.

Why it matters

- **Safety:** functions can't alter strings you pass them — no defensive copies.
- **Hashability:** a fixed value means a fixed hash (an integer fingerprint used for fast lookup), so strings can be dict keys and set members (Chapter 6). CPython caches a string's hash after the first computation.
- **Sharing:** the interpreter may reuse one object in many places (identifiers are *interned* — stored once and shared), safe only because nobody can mutate it.

How it works

Item assignment → `TypeError`. To "update", slice what you keep, concatenate the new text, and rebind the name. The old object is untouched (garbage-collected if unreferenced).

```
s = "Hello, world!"
try:
    s[0] = "J"                          # not allowed
except TypeError as e:
    print("TypeError:", e)
before = id(s)
s = s[:7] + "Python!"                   # build a NEW string, rebind the name
print(s, "| same object?", id(s) == before)
```

Output:

```
TypeError: 'str' object does not support item assignment
Hello, Python! | same object? False
```

`id()` differs: a new object was created.

Trade-offs

	Immutable `str`	**Mutable alternative (`list` of chars, `bytearray`, `io.StringIO`)**
Safe to share / dict key	Yes	No
Change one character	O(n) — copy the whole string	O(1)
Build text piece by piece	`"".join(parts)` (4.8)	Natural

Key term — Big-O notation: how cost grows with input size *n*. **O(1)**: constant; **O(n)**: grows in step with length; **O(n²)** ("quadratic"): doubling *n* quadruples the work. It describes growth, not timings.

Common mistakes

- Ignoring the result: `name.strip()` alone does nothing. Write `name = name.strip()`.
- Thinking `s += "x"` modifies in place — it creates a new string (4.8).

In the interview

30-second answer: "Python strings are immutable: every 'modifying' operation returns a new string. That makes them safe to share and hashable, so they work as dict keys. The cost is that repeated modification copies data, so for building large strings you collect parts in a list and `join` them."

- *How does `s += 'x'` work?* → Builds `s + 'x'` and rebinds `s`. (CPython sometimes resizes in place when no other reference exists — don't rely on it.)
- *Why must dict keys be immutable?* → Strictly, *hashable*: a changing hash would make lookups search the wrong slot.

TL;DR

You never change a string; you make a new one and rebind. Immutability buys safety and hashability.

4.3 String operators

What it is

Operator	Meaning	Example	Result
`+`	Concatenation	`"Py" + "thon"`	`'Python'`
`*`	Repetition (int times)	`"Ha" * 3`	`'HaHaHa'`
`in`	Substring membership	`"fox" in "The quick brown fox"`	`True`
`not in`	Negated membership	`"cat" not in "The quick brown fox"`	`True`
`==`, `!=`	Equality by value	`"Python" == "Python"`	`True`
`<`, `>`, `<=`, `>=`	Lexicographic (dictionary-order) comparison	`"apple" < "banana"`	`True`
`%`	printf-style formatting (from C's `printf`; legacy, 4.6)	`"%s!" % "hi"`	`'hi!'`
`[i]`, `[a:b:c]`	Indexing and slicing (4.4)	`"abc"[1]`	`'b'`

How it works

1. `+` needs **two strings**: `"Age: " + 30` → `TypeError`; use `str(30)` or an f-string.
2. `*` with zero or negative gives `''`.
3. `in` tests **substrings**: `"row" in "brown"` is `True`.
4. Comparisons go character by character by **Unicode code point** (4.9); first difference decides; a prefix is smaller (`"app" < "apple"`).
   - `"Zebra" < "apple"`: `'Z'` (90) < `'a'` (97) → `True`. Uppercase sorts before lowercase.
   - `"10" < "9"` is `True` (`'1'` 49 < `'9'` 57) — numeric strings don't compare as numbers.

```
print("Ha" * 3, "+".join(["a", "b"]), "Py" + "thon")
print("fox" in "The quick brown fox", "cat" not in "The quick brown fox")
print("apple" < "banana", "Zebra" < "apple", "10" < "9")  # compares code points
```

Output:

```
HaHaHa a+b Python
True True
True True True
```

Common mistakes

- Sorting names with plain `<` (`"Zoe"` before `"adam"`). Use `key=str.casefold` (a *key function* decides each item's sort position).
- Comparing numeric strings — convert with `int()`/`float()`.
- `is` for strings checks identity; it may "work" for short interned strings then fail. Use `==`.

TL;DR

`+` joins, `*` repeats, `in` finds substrings, comparisons are by code point (case and digits matter), use `==` not `is`.

4.4 Slicing

What it is

`s[start:stop:step]` returns a new string:

- `start` — first position included (default: beginning).
- `stop` — first position **excluded** (default: end).
- `step` — jump size (default `1`; negative walks backwards).

The same syntax works on lists, tuples, bytes and arrays (Chapter 5): fixed-position fields, dropping prefixes, reversing, every *n*-th item.

How it works

1. Negatives get `len(s)` added (`len("Hello, Python!") == 14`, so `-7` → `7`).
2. Missing parts take defaults.
3. Walk from `start` toward `stop` by `step`, stopping **before** `stop`.
4. Out-of-range bounds are clipped — slicing never raises `IndexError`.

Half-open `[start, stop)` gives `len(s[a:b]) == b - a` and `s[:k] + s[k:] == s`.

```
text = "Hello, Python!"
         H  e  l  l  o  ,  ␣  P  y  t  h  o  n  !
index:   0  1  2  3  4  5  6  7  8  9 10 11 12 13
neg:   -14                   -7 -6 -5 -4 -3 -2 -1
```

```
text = "Hello, Python!"
print(repr(text[7:13]))    # 'Python'
print(repr(text[-7:-1]))   # 'Python'
print(repr(text[:5]), repr(text[7:]))
print(repr(text[::2]))     # every 2nd char: 'Hlo yhn'
print(repr(text[::-1]))    # reversed
print(repr(text[3:100]))   # slices never raise IndexError -> clipped
```

Output:

```
'Python'
'Python'
'Hello' 'Python!'
'Hlo yhn'
'!nohtyP ,olleH'
'lo, Python!'
```

`text[::2]` takes positions 0, 2, … 12 → `H l o ␣ y h n`; the comma (odd position 5) is skipped. `text[-5:-1]` is `'thon'`.

Common mistakes

- Expecting `stop` included: `text[7:12]` is `'Pytho'`.
- Negative step: `s[5:1:-1]` walks 5 **down to** 2; `s[1:5:-1]` is empty.
- `[::-1]` reverses code points, which can detach combining accents or split emoji with modifiers (skin tones) — 4.9.

In the interview

30-second answer: "`s[start:stop:step]` returns a new sequence from `start` up to but not including `stop`. Omitted bounds default to the ends, negatives count from the end, bounds are clipped so it never raises, and `s[::-1]` reverses. It costs O(k) for k items copied."

- *Why is `stop` exclusive?* → `s[:k] + s[k:] == s`, `len(s[a:b]) == b - a`, no ±1 corrections.

- *Palindrome check* `s == s[::-1]`*?* → O(n) time and O(n) extra space; a two-pointer loop (indexes moving in from both ends) uses O(1) space.

Self-check

1. `"abcdef"[::3]`? → `'ad'`. 2. `"abc"[5:]`? → `''`.

TL;DR

`[start:stop:step]`, stop excluded, negatives from the end, never raises, `[::-1]` reverses, cost ∝ slice length.

4.5 Escape sequences and raw strings

What it is

An **escape sequence** is a backslash plus characters standing for one special character (`\n` = newline). A **raw string** (`r` prefix) disables escapes — every backslash is literal. Escapes put newlines, tabs, quotes and any Unicode into literals; raw strings suit backslash-heavy text: regular expressions (§4.10) and Windows paths.

How it works

Escape	Meaning	Example	Result
`\\`	Backslash	`"a\\b"`	`a\b`
`\'`	Single quote	`'It\'s'`	`It's`
`\"`	Double quote	`"He said \"hi\""`	`He said "hi"`
`\n`	Newline (line feed)	`"one\ntwo"`	two lines
`\t`	Horizontal tab	`"a\tb"`	`a` tab `b`
`\r`	Carriage return (cursor to line start)	`"Hello\rWorld"`	terminal shows `World`
`\b`	Backspace character (code 8)	`"ABC\b"`	terminal may erase `C` visually
`\f`	Form feed (printer page break)	`"Page\fNext"`	rarely visible
`\ooo`	Octal code (up to 3 digits)	`"\101"`	`A`
`\xhh`	Hex code (exactly 2 digits)	`"\x41"`	`A`
`\uXXXX`	Code point (4 hex digits)	`"\u00e9"`	`é`
`\UXXXXXXXX`	Code point (8 hex digits)	`"\U0001F600"`	😀
`\N{name}`	Character by Unicode name	`"\N{GREEK SMALL LETTER PI}"`	`π`

`\r` and `\b` are real characters; what you *see* depends on the terminal (`repr("ABC\b")` is `'ABC\x08'`).

```
print("Tab\tseparated\nnew line, quote: \" backslash: \\")
print("\x41\101\u00e9\N{GREEK SMALL LETTER PI}")   # A A é π
path = r"C:\new\table"                              # raw: backslashes kept literally
print(path, len(path), len("C:\new\table"))         # raw has 12 chars, cooked has 10
```

Output:

```
Tab	separated
new line, quote: " backslash: \
AAéπ
C:\new\table 12 10
```

Without `r`, `"C:\new\table"` silently contains a newline and a tab.

Common mistakes

- Windows paths without `r` — or better, use forward slashes or `pathlib.Path`.
- Unknown escapes like `"\d"`: kept, but `SyntaxWarning` since 3.12. Write regexes raw: `r"\d+"` (`"\b"` without `r` is a backspace, not a word boundary).
- A raw string can't end in one backslash: `r"C:\"` is a syntax error. Use `"C:\\"`.

TL;DR

Backslash escapes produce special characters; `r"..."` disables them — use it for regexes and Windows paths.

4.6 String formatting: f-strings, the format-spec mini-language, `str.format`, `%`

What it is

Formatting builds a string from a template plus values.

Style	Example	Introduced	Use today
f-string (formatted string literal)	`f"{name} is {age}"`	3.6	Default choice
`str.format()`	`"{} is {}".format(name, age)`	2.6 / 3.0	Templates stored as data
`%` (printf-style)	`"%s is %d" % (name, age)`	Original	Legacy code, `logging` calls
`string.Template`	`Template("$name is $age")`	2.4	Templates written by untrusted users

Python 3.14 added **t-strings** (`t"..."`), producing a `string.templatelib.Template` object (unrelated to `string.Template`) instead of a `str`, so libraries can escape values safely (e.g. for HTML or SQL). Still niche; f-strings remain the everyday tool.

Why it matters

Reports, log lines, money with two decimals, padded columns, hex or percent. The **format-spec mini-language** after `:` is shared by f-strings, `str.format()` and `format()` — learn it once.

How it works — f-strings

1. Prefix `f`; inside `{ }` put **any expression**.
2. Optional **conversion**: `!r` (`repr()`, the unambiguous developer form), `!s` (`str()`), `!a` (`ascii()`).
3. Optional `:format_spec`.
4. `{expr=}` (3.8+) prints the expression text *and* its value — a debugging shortcut.
5. `{{` / `}}` for literal braces.

```python
name, age = "Asha", 31
print(f"{name} is {age} years old.")
print(f"Next year: {age + 1}, upper: {name.upper()}")
print(f"{name!r} vs {name!s}")                # !r calls repr(), !s calls str()
width = 10
print(f"[{name:>{width}}]")                    # nested replacement field for the width
total = 42
print(f"{total = }")                           # self-documenting (3.8+)
print(f"{{literal braces}}")
```

Output:

```
Asha is 31 years old.
Next year: 32, upper: ASHA
'Asha' vs Asha
[      Asha]
total = 42
{literal braces}
```

Since 3.12 (PEP 701) f-strings are parsed by the real grammar: you can reuse the same quote type inside braces (`f"{d["key"]}"`), use backslashes, and split expressions over lines. On ≤ 3.11 use the other quote type inside.

How it works — the format-spec mini-language

```
[[fill]align][sign][#][0][width][grouping][.precision][type]
```

Part	Options	Meaning
fill	any character	Pad character (needs an align)
align	`<` `>` `^` `=`	Left, right, centre, pad after the sign
sign	`+` `-` space	Always sign / only minus (default) / space for positives
`#`		Alternate form: `0x`, `0o`, `0b` prefixes
`0`		Zero-pad numbers
width	integer	Minimum total width
grouping	`,` or `_`	Thousands separator
.precision	integer	Digits after the point (`f`, `%`), significant digits (`g`), max chars (strings)
type	`s` `d` `b` `o` `x` `X` `c` `e` `f` `g` `%`	String, decimal, binary, octal, hex, char, scientific (`1.2e+06`), fixed, general, percent

Defaults: strings left-aligned, numbers right-aligned.

Output of `code/02_formatting.py` (spec → result; `pi, n, ratio = 3.14159265, 1234567, 0.4567`):

```
{:.2f}                -> '3.14'
{:10.3f}              -> '     3.142'
{:<8}|                -> 'ab      |'
{:^8}|                -> '   ab   |'
{:*>8}                -> '******ab'
{:,}                  -> '1,234,567'
{:_}                  -> '1_234_567'
{:.1%}                -> '45.7%'
{:e}                  -> '1.234567e+06'
{:+d}                 -> '+42'
{:05d}                -> '00042'
{:b} {:o} {:x} {:X}   -> '11111111 377 ff FF'
{:#x}                 -> '0xff'
{:%Y-%m-%d}           -> '2024-03-09'
```

`f"{pi:10.3f}"`: fixed point, 3 decimals → `3.142` (5 chars), width 10, right-aligned → `' 3.142'`.

The last row: `date` passes everything after `:` to `strftime` (formatting with codes like `%Y`). Your classes can do the same by implementing `__format__`.

How it works — `str.format()` and `%`

```
print("{} + {} = {}".format(2, 3, 5))
print("{1} before {0}".format("A", "B"))
print("{name} has {n} cats".format(name="Li", n=2))
user = {"name": "Li", "n": 2}
print("{name} has {n} cats".format(**user))
template = "Dear {who}, your balance is {amt:,.2f}"   # reusable template stored as data
print(template.format(who="Sam", amt=12345.6))
print("%s is %d years, pi~%.2f" % ("Asha", 31, 3.14159))  # old printf style
```

Output:

```
2 + 3 = 5
B before A
Li has 2 cats
Li has 2 cats
Dear Sam, your balance is 12,345.60
Asha is 31 years, pi~3.14
```

- `{}` positional, `{1}` by index, `{name}` by keyword; `**user` unpacks a dict.
- `str.format()` shines when the **template is data** (from a file or config); f-strings must be in source.
- `%`: `%s`, `%d`, `%f`, `%x`, `%%`. A single tuple value misbehaves — one reason newer styles won.

Trade-offs

	f-string	str.format	%	string.Template
Readability	Best	Good	Fair	Good
Speed	Fastest (compiled into the function)	Slower (call + parse)	Similar to format	Slowest
Template can be data	No	Yes	Yes	Yes
Safe for user-written templates	n/a	**No** — `{obj.__class__...}` reaches attributes	Mostly	**Yes** — only `$name` substitution
Lazy formatting in logging	No	No	Yes (`logger.info("%s", x)`)	No

When not to use it

- **Logging:** `logger.info("user %s logged in", user)`, not an f-string. `logging` formats only if the level is enabled, and log aggregators (services collecting logs from many machines) group by the unformatted template.
- **SQL:** never format values into SQL. Use driver parameters (`cursor.execute("... WHERE id = ?", (user_id,))`) to prevent SQL injection (user input executed as database commands).
- **User-supplied templates:** `user_template.format(**objects)` can walk attributes (`{config.__class__.__init__.__globals__}`) and leak data. Use `string.Template` or a sandboxed template engine.

Common mistakes

- Forgetting `f` → braces print literally.

- `f"{price:.2}"` = **2 significant digits**, not two decimals; use `:.2f` (and `decimal.Decimal` for real money).
- `"%s" % some_tuple` → "not all arguments converted"; write `"%s" % (some_tuple,)`.

Real-world examples

- `logging` merges message arguments `%`-style; `style="{"` changes only the *Formatter's* layout string, not messages.
- `datetime`, `Decimal`, `ipaddress` implement `__format__`: `f"{now:%H:%M}"`.
- Config and email templates stored as data, filled with `str.format` or a template engine such as Jinja2.

In the interview

> **30-second answer:** "Use f-strings by default: readable and fastest. After the colon is the format-spec mini-language — fill, align, sign, width, grouping, precision, type — e.g. `{x:>10,.2f}`. Use `str.format` when the template is stored as data, `%` for logging's lazy formatting, and `string.Template` for untrusted templates."

- *`f"{x=}`? → `x=` followed by `repr(x)` (3.8).
- *Support `f"{obj:custom}`? → Implement `__format__(self, spec)`.
- *Why not f-strings in log calls?* → Built even when the level is disabled; prevents grouping by template.

Self-check

1. `f"{7:03d}"`? → `'007'`. 2. `f"{0.256:.0%}"`? → `'26%'`. 3. `f"{'hi':=^6}"`? → `'==hi=='`.

TL;DR

f-strings everywhere; learn `[[fill]align][sign][#][0][width][,][.prec][type]`; `str.format` for stored templates; `%` for logging; `Template` for untrusted input.

4.7 String methods

What it is

A **method** is a function attached to an object (`text.upper()`). `str` has about 47. None change the original: transformers **return a new string**, checkers return `True`/`False` or a number. They run in optimised C — far faster than hand-written loops for cleaning CSV fields, validating input, parsing logs.

How it works — the reference table

Case conversion

Method	Returns
`upper()` / `lower()`	All letters upper / lower
`capitalize()`	First character upper, rest lower
`title()`	First letter of each word upper, rest lower
`swapcase()`	Each letter's case flipped
`casefold()`	Aggressive lowercase for caseless comparison (`"ß"` → `"ss"`)

Alignment and padding

Method	Returns
`center(w, fill=" ")`	Centred in width `w`
`ljust(w, fill)` / `rjust(w, fill)`	Left / right justified
`zfill(w)`	Left-padded with zeros (keeps a leading sign)
`expandtabs(n)`	Tabs replaced by spaces

Stripping, splitting, joining

Method	Returns
`strip(chars)` / `lstrip` / `rstrip`	Remove whitespace (or any of `chars`) from both ends / left / right
`split(sep=None, maxsplit=-1)`	List of pieces; no `sep` → split on whitespace runs, drop empties
`rsplit(sep, maxsplit)`	Like `split`, counting `maxsplit` from the right
`splitlines()`	Lines (`\n`, `\r\n`, others) without breaks
`partition(sep)` / `rpartition(sep)`	`(before, sep, after)` around first / last `sep`
`sep.join(iterable)` (any loopable object)	Items (must be strings) glued with `sep`
`removeprefix(p)` / `removesuffix(s)`	Without that exact prefix/suffix, if present (3.9+)

Boolean checks (all `False` for `""`, except `isprintable()` and `isascii()`)

Method	`True` when…
`isalnum()`	Every character is a letter or number (Unicode sense)
`isalpha()`	Every character is a letter
`isdecimal()`	Every character is a decimal digit 0–9 (in any script, e.g. Arabic-Indic)
`isdigit()`	Decimal digits plus digits like superscript `²`
`isnumeric()`	Any numeric character, including `½`
`islower()` / `isupper()`	All cased characters lower / upper, at least one
`istitle()`	Upper only after uncased chars, lower only after cased ones
`isspace()`	Only whitespace, at least one
`isidentifier()`	Valid Python identifier (keywords included — check `keyword.iskeyword` too)
`isprintable()`	No non-printables (`\t`, `\n` count as non-printable); `True` for `""`
`isascii()`	All ASCII (the 128-character English set; 3.7+); `True` for `""`
`startswith(p)` / `endswith(s)`	Starts / ends with; accepts a **tuple** of options

Find, count, replace, translate

Method	Returns
`count(sub, start, end)`	Non-overlapping occurrences
`find(sub, start, end)` / `rfind`	Lowest / highest index, or `-1`
`index(sub, start, end)` / `rindex`	Same, but raises `ValueError`
`replace(old, new, count=-1)`	Copy with occurrences replaced (all, or first `count`)
`str.maketrans(...)` + `translate(table)`	Map/delete many single characters in one pass

Optional `start, end` (`beg`, `end` in older docs) search inside `s[start:end]` but report indexes in the full string.

Code — each group in action

```
s = "pYthon is FUN"
print(s.upper(), "|", s.lower(), "|", s.capitalize(), "|", s.title(), "|", s.swapcase())
print("they're bill's".title())           # title() treats ' as a word break
print("Straße".lower(), "Straße".casefold())  # casefold is the aggressive, caseless form
print("[" + "Menu".center(16, "-") + "]")
print("[" + "Menu".ljust(10) + "]", "[" + "Menu".rjust(10, ".") + "]", "42".zfill(5))
```

Output:

```
PYTHON IS FUN | python is fun | Python is fun | Python Is Fun | PyTHON IS fun
They're Bill'S
straße strasse
[------Menu------]
[Menu      ] [......Menu] 00042
```

With odd padding, which side of `center` gets the extra character depends on the lengths (`"ab".center(5)` is `' ab '`, `"abc".center(6)` is `' abc '`), whereas f-string `{:^w}` always puts it on the right. Pick one if columns must line up.

```
text = "  Hello, Python! Welcome to programming.  "
print(text.strip().split())               # any whitespace, runs collapsed
print(text.strip().rsplit(" ", 1))        # split once, from the right
print("a,,b".split(","), "a  b".split(" "), "a  b".split())
print("k=v=w".partition("="), "k=v=w".rpartition("="), "abc".partition("="))
print("line1\nline2\r\nline3".splitlines())
print("xxhixx".strip("x"), "--hi".lstrip("-"), "hi--".rstrip("-"))
print("report_2024.csv".removesuffix(".csv"), "Mr. Smith".removeprefix("Mr. "))
print("abcabc".strip("ab"))               # strip removes a SET of chars, not a prefix
print("-".join(["2024", "03", "09"]), ", ".join(str(n) for n in [1, 2, 3]))
```

Output:

```
['Hello,', 'Python!', 'Welcome', 'to', 'programming.']
['Hello, Python! Welcome to', 'programming.']
['a', '', 'b'] ['a', '', 'b'] ['a', 'b']
('k', '=', 'v=w') ('k=v', '=', 'w') ('abc', '', '')
['line1', 'line2', 'line3']
hi hi hi
report_2024 Smith
cabc
2024-03-09 1, 2, 3
```

Memorise:

- `split()` with **no argument** collapses whitespace runs and never yields empties; `split(" ")` yields `''` between adjacent separators.
- `strip("ab")` removes **any** of `a`/`b` from the ends — not the prefix `ab`; that's `removeprefix`.

```
samples = ["Hello123", "Hello", "123", "½", "²", "   ", "Hello World", "my_var", "3var"]
# prints one row per sample: isalnum isalpha isdecimal isdigit isnumeric isspace istitle isidentifier
```

Output (T = True, F = False):

```
value          alnum alpha decim digit numer space title ident
'Hello123'         T     F     F     F     F     F     T     T
'Hello'            T     T     F     F     F     F     T     T
'123'              T     F     T     T     T     F     F     F
'½'                T     F     F     F     T     F     F     F
'²'                T     F     F     T     T     F     F     F
'   '              F     F     F     F     F     T     F     F
'Hello World'      F     F     F     F     F     F     T     F
'my_var'           F     F     F     F     F     F     F     T
'3var'             T     F     F     F     F     F     F     F
True True
```

Nested sets: **decimal ⊂ digit ⊂ numeric**. To validate "can `int()` take this?", use `isdecimal()` (`int("²")` fails though `"²".isdigit()` is `True`). With signs, spaces or decimals, just `try: int(s)` / `float(s)`.

```python
t = "hello world, hello universe, hello multiverse"
print(t.count("hello"), t.find("hello"), t.rfind("hello"), t.find("bye"))
print(t.find("hello", 1))                 # search starting at index 1
try:
    t.index("bye")
except ValueError as e:
    print("ValueError:", e)
print(t.replace("hello", "hi", 2))        # only first 2 occurrences

table = str.maketrans({"a": "4", "e": "3", "o": "0", "!": None})  # None = delete
print("hello, leet coders!".translate(table))
```

Output:

```
3 0 29 -1
13
ValueError: substring not found
hi world, hi universe, hello multiverse
h3ll0, l33t c0d3rs
```

Cost of common operations (n = length of the string)

Operation	Time	Note
`len(s)`	O(1)	Length is stored
`s[i]`	O(1)	Fixed-width storage (4.9)
`s[a:b]`	O(b − a)	Copies the slice
`s + t`	O(len s + len t)	Always a new string
`sub in s`, `find`, `count`, `replace`	O(n) typical	Optimised C search
`split`, `strip`, `upper`, `translate`	O(n)	One pass
`sep.join(parts)`	O(total length)	Pre-computes size, one copy
`hash(s)`	O(n) first time, then O(1)	Cached on the object
`s == t`	O(n) worst case	Stops at first difference; O(1) if lengths differ

Common mistakes

- `if s.find("x"):` is **wrong** — `0` (match at start) is falsy, `-1` (absent) truthy. Use `if "x" in s:`.
- `lower()` for international caseless comparison — prefer `casefold()`.
- `title()` mangles apostrophes (`"They'Re"`); `string.capwords()` splits on whitespace only.

- `",".join([1, 2])` → `TypeError`; use `",".join(map(str, nums))` (`map` applies `str` to each item lazily, like a generator — Chapter 7).

Real-world examples

- `line.rstrip("\n").split(",")` in countless scripts (use `csv` when fields contain quoted commas).
- `url.removeprefix("https://")`, `filename.endswith((".jpg", ".png"))`.
- Search boxes normalise with `query.strip().casefold()`.

In the interview

> **30-second answer:** "String methods never mutate; they return new strings. Key groups: case (`lower`, `casefold`), cleaning (`strip`, `removeprefix`), splitting/joining (`split`, `partition`, `join`), checking (`isdecimal`, `startswith` with tuples) and searching (`in`, `find` returns −1, `index` raises). Most are O(n) and implemented in C."

- *`find` vs `index`?* → `-1` vs `ValueError` when absent.
- *`split()` vs `split(" ")`?* → No-arg collapses whitespace runs and drops empty fields.
- *Remove all punctuation fast?* → `s.translate(str.maketrans("", "", string.punctuation))`.

Self-check

1. `"a-b-c".rsplit("-", 1)`? → `['a-b', 'c']`. 2. `"Menu".center(20, "-")`? → `'--------Menu--------'`.

TL;DR

Methods return new strings. Memorise `strip/split/join/partition`, `startswith(tuple)`, `in` over `find`, `casefold` for comparisons, `isdecimal` for digits, `translate` for bulk mapping.

4.8 Joining strings efficiently

What it is

To build a big string from many pieces, collect them in a list and call `"".join(pieces)` once.

The problem it solves

Immutability means `s = s + piece` allocates a new string and copies **all** existing characters each time: 1 + 2 + … + n = n(n+1)/2 copies — **quadratic**, O(n²). `join` sums the lengths, allocates once, copies each piece once — **linear**, O(n).

How it works

1. Append pieces to a list (amortised O(1) — occasionally slow, constant on average; Chapter 5).
2. `sep.join(list)`: pass 1 adds up lengths; pass 2 copies into one pre-sized buffer.
3. Stream-style alternative: `io.StringIO` (in-memory text file) — `write()` pieces, then `getvalue()`.

CPython's trick: if the left string of `+=` has **no other references**, it may resize it in place, so the naive loop looks fast. Implementation detail — PyPy doesn't do it, and a second reference defeats it. The benchmark shows both:

```python
def plus_equal(n):          # naive, but CPython's in-place trick rescues it
    s = ""
    for _ in range(n):
        s += PIECE
    return s

def plus_equal_shared(n):   # same, but a second reference defeats the trick
    s = ""
    alias = []
    for _ in range(n):
        alias.append(s)
        s += PIECE
        alias.pop()
    return s

def join_list(n):
    parts = []
    for _ in range(n):
        parts.append(PIECE)
    return "".join(parts)
```

Output (`code/05_join_performance.py`, ms on the author's laptop — yours differ, the shape won't):

```
     n         plus_equal  plus_equal_shared         join_list          string_io
  2000            0.07 ms            0.23 ms           0.04 ms            0.04 ms
  5000            0.16 ms            0.89 ms           0.09 ms            0.10 ms
 10000            0.34 ms            3.06 ms           0.17 ms            0.20 ms
 20000            0.69 ms           18.84 ms           0.35 ms            0.41 ms
 40000            1.36 ms           81.27 ms           0.69 ms            0.82 ms
```

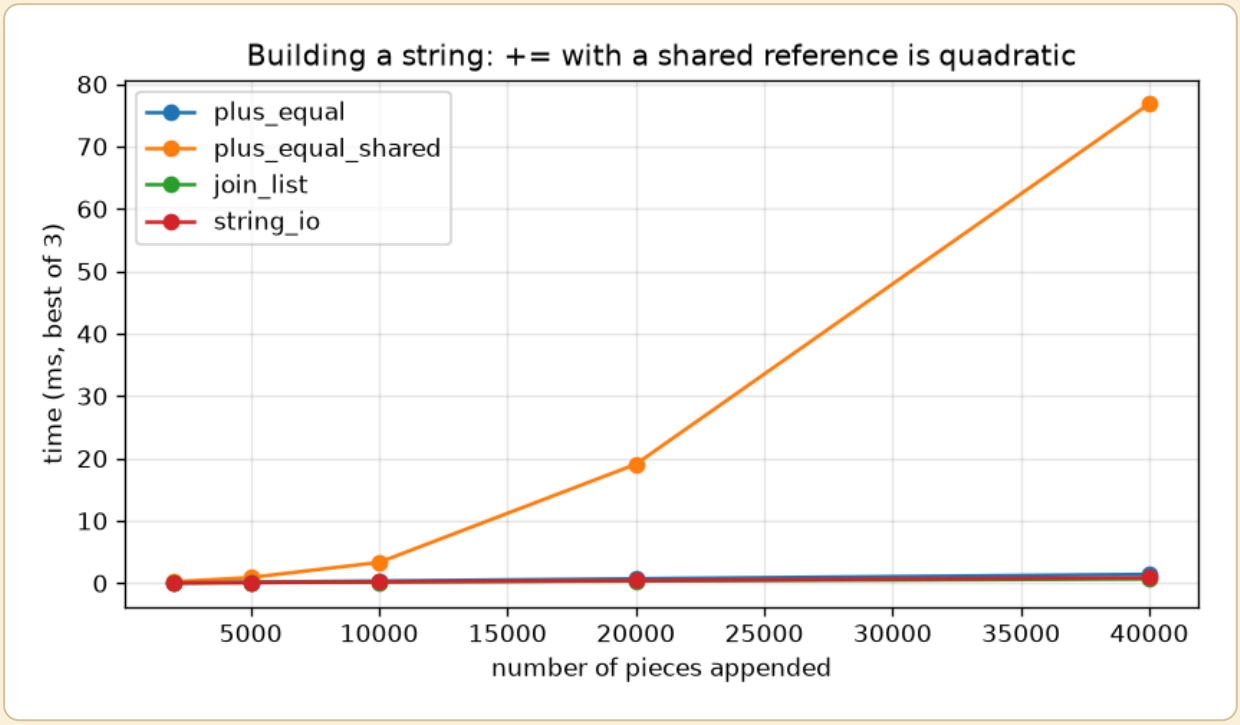


Figure — doubling n doubles `join`/`StringIO` time (linear) but roughly quadruples `+=` when the optimisation can't apply (quadratic).

Trade-offs

Approach	Complexity	Readability	Notes
`a + b + c` (few pieces)	Fine	Best	OK for a handful
f-string	Fine	Best	Fixed templates
`+=` in a loop	O(n²) worst case	Good	Fast only by CPython accident
`"".join(list_or_generator)`	O(n)	Good	The standard idiom
`io.StringIO`	O(n)	Good	When code "writes" output in many places

When not to use it

Two or three concatenations: `first + " " + last` or an f-string is clearer. The join rule is for loops and large inputs.

Common mistakes

- Joining numbers (`TypeError`) — `map(str, nums)`.
- `"".join([x for x in items])` when `"".join(items)` works.

In the interview

> **30-second answer:** "Repeated `+=` on an immutable string copies everything each time, so it's O(n²). Collect parts in a list and `''.join` once — O(n), because join sizes the result first and copies each part once."

- *My `+=` benchmark is fast?* → CPython resizes in place for single-reference strings; not guaranteed or portable.
- *Generator or list in `join`?* → `join` converts to a sequence anyway, so a list comprehension is slightly faster; both fine.

TL;DR

Many pieces → list + `"".join(...)` (or `StringIO`). A few → `+` or f-string.

4.9 Unicode vs bytes, encodings and UTF-8

What it is

- A **character** is an abstract symbol: "A", "é", "€", "😀".
- **Unicode** assigns each a **code point**, written `U+20AC` for "€". `str` is a sequence of code points; `ord(ch)` / `chr(n)` convert.
- **Bytes** are raw 8-bit values (0–255) — what files, sockets and disks hold. `bytes` is immutable, written `b"..."`.
- An **encoding** turns code points into bytes (`str.encode`) and back (`bytes.decode`).
- **UTF-8** dominates the web and Linux/macOS: variable-length, 1–4 bytes per code point, identical to ASCII for the first 128.

Read with the wrong encoding, text becomes gibberish ("mojibake").

Why it matters

Every boundary — reading a file, calling an HTTP API, writing to a database — converts between `str` and `bytes`. A wrong encoding crashes (`UnicodeDecodeError`) or, worse, silently corrupts text.

How it works

1. Work with `str` inside the program.
2. **Decode** incoming bytes early, **encode** outgoing text late (the "Unicode sandwich").
3. Name the encoding: `open(path, encoding="utf-8")`. Otherwise `open()` uses the locale encoding (OS language/region default), which varies by machine. Python 3.15 (due October 2026) makes UTF-8 the default (PEP 686) — still name it while supporting 3.12–3.14.
4. UTF-8 lengths:

Code points	Bytes	Example
U+0000 – U+007F (ASCII)	1	`A` → `41`
U+0080 – U+07FF	2	`é` → `C3 A9`
U+0800 – U+FFFF	3	`€` → `E2 82 AC`
U+10000 – U+10FFFF	4	`😀` → `F0 9F 98 80`

```
for ch in ["A", "é", "€", "😀"]:
    utf8 = ch.encode("utf-8")
    print(f"{ch!r}: ord={ord(ch):>6}  U+{ord(ch):04X}  utf-8={utf8!r:<22} {len(utf8)} byte(s)")

s = "café"
b = s.encode("utf-8")
print(type(b), b, "len(str)=", len(s), "len(bytes)=", len(b))
print(b.decode("utf-8"))
print(repr(b.decode("latin-1")))                    # wrong codec -> mojibake, no error!
try:
    b.decode("ascii")
except UnicodeDecodeError as e:
    print("UnicodeDecodeError:", e)
print(s.encode("ascii", errors="replace"), s.encode("ascii", errors="ignore"))
print(b"\xff\xfeok".decode("utf-8", errors="replace"))
print(b"abc"[0], list(b"hi"), bytes([104, 105]))        # indexing bytes gives ints
```

Output:

```
'A': ord=    65  U+0041  utf-8=b'A'                  1 byte(s)
'é': ord=   233  U+00E9  utf-8=b'\xc3\xa9'           2 byte(s)
'€': ord=  8364  U+20AC  utf-8=b'\xe2\x82\xac'       3 byte(s)
'😀': ord=128512  U+1F600  utf-8=b'\xf0\x9f\x98\x80'    4 byte(s)
<class 'bytes'> b'caf\xc3\xa9' len(str)= 4 len(bytes)= 5
café
'cafÃ©'
UnicodeDecodeError: 'ascii' codec can't decode byte 0xc3 in position 3: ordinal not in range(128)
b'caf?' b'caf'
��ok
97 [104, 105] b'hi'
```

- `"café"` is 4 characters but 5 UTF-8 bytes; database column limits and HTTP `Content-Length` count **bytes**.
- Decoding UTF-8 as Latin-1 (an old 256-character Western-European encoding) **doesn't fail** — it yields `cafÃ©`, classic mojibake.
- `errors=`: `"strict"` (default, raise), `"replace"` (`?` / `�`), `"ignore"` (drops data silently), `"backslashreplace"`, more.
- Indexing `bytes` gives an **int**.

Normalization. `é` can be one code point (U+00E9) or `e` + combining accent (U+0301): identical-looking, unequal. `unicodedata.normalize("NFC", s)` converts to the composed form:

```
a = "caf\u00e9"; b = "cafe\u0301"
print(a, b, a == b, len(a), len(b))          # café café False 4 5
nfc = unicodedata.normalize("NFC", b)
print("after NFC:", a == nfc, len(nfc))      # after NFC: True 4
```

How CPython stores `str`. Since 3.3 (PEP 393) each string uses the narrowest fixed width fitting its widest character: 1 byte/char for Latin-1, 2 up to U+FFFF, 4 otherwise. Fixed width makes `s[i]` O(1) — CPython does not store UTF-8 internally.

Trade-offs

	`str`	`bytes`	`bytearray`
Holds	Code points	Raw bytes 0–255	Raw bytes 0–255
Mutable	No	No	Yes
Literal	`"héllo"`	`b"hello"` (ASCII only in literal)	`bytearray(b"...")`
Use for	All text logic	Files, network, hashing, cryptography	Building/patching binary buffers

Encoding	Size per char	ASCII-compatible	Notes
UTF-8	1–4 bytes	Yes	Web, JSON (required by RFC 8259 for data exchanged between systems), Python source
UTF-16	2 or 4 bytes	No	Windows APIs, Java and JavaScript strings
Latin-1 (ISO-8859-1)	1 byte	Yes	256 characters; never fails to decode, so hides errors
ASCII	1 byte	—	128 characters

Common mistakes

- `"abc" + b"def"` → `TypeError`; Python 3 never guesses an encoding.
- Omitting `encoding=` in `open()` — works on your Mac, breaks on a Windows server.
- `len()` for user-visible characters: a flag or "👍" is several code points. "Grapheme" (perceived character) counting needs `grapheme` or `regex` (`\X`).
- `errors="ignore"` to silence an error — silent data loss.

Real-world examples

- `requests`: `response.content` (bytes) vs `response.text` (decoded using the declared or guessed encoding).
- `json.dumps(..., ensure_ascii=False)` keeps non-ASCII readable instead of `\u00e9`.
- Python source is UTF-8 by default (PEP 3120), so `name = "Zoë"` needs no `# -*- coding: … -*-` comment.

In the interview

> **30-second answer:** "`str` is a sequence of Unicode code points; `bytes` is raw data. Encode text to bytes at the output boundary and decode bytes to text at the input boundary, always naming the encoding — normally UTF-8, which is variable-length (1–4 bytes) and ASCII-compatible."

- *`len('café'.encode())` is 5?* → `é` needs two bytes.
- *Mojibake?* → Text decoded with the wrong encoding (UTF-8 read as Latin-1 → `cafÃ©`).
- *Equal-looking strings unequal?* → Composed vs decomposed code points; normalise with NFC.

Self-check

1. Bytes for `€` in UTF-8? → 3. 2. Can `bytes.decode("latin-1")` raise? → No — every byte maps, which hides mistakes.

TL;DR

Text is `str` (code points); storage and network are `bytes`. Decode early, encode late, always say `utf-8`, normalise before comparing user text.

4.10 Regular expressions — a practical introduction

What it is

A **regular expression** (regex) is a pattern language for text *shapes*: "one or more digits", "a date like 2024-03-09". `re` compiles a pattern and searches strings. String methods handle fixed text; regex makes extracting emails, validating postcodes, parsing log fields or collapsing whitespace one-liners.

How it works — the core syntax

Pattern	Matches
`.`	Any character except newline
`\d` `\w` `\s`	Digit, word char (letter, digit, `_`), whitespace
`\D` `\W` `\S`	The opposites
`[abc]` `[a-z]` `[^0-9]`	One char from a set / range / not in the set
`^` `$`	Start / end of string (each line with `re.MULTILINE`); `$` also matches before a final `\n`
`\b`	Word boundary
`*` `+` `?`	0+, 1+, 0 or 1 (greedy — as much as possible)
`{3}` `{2,5}`	Exactly 3, 2 to 5
`*?` `+?`	Lazy — as little as possible
`a|b`	`a` or `b`
`( … )`	Capturing group (remembers its text)
`(?P<name> … )`	Named group
`(?: … )`	Non-capturing group

The main functions

Function	Does
`re.search(p, s)`	First match **anywhere** → `Match` or `None`
`re.match(p, s)`	Match only at the **start**
`re.fullmatch(p, s)`	**Whole string** must match
`re.findall(p, s)`	All matches (strings, or tuples with several groups)
`re.finditer(p, s)`	Iterator of `Match` objects (positions + groups)
`re.sub(p, repl, s)`	Replace; `repl` may use `\1` (a *backreference* to group 1's text) or be a function
`re.split(p, s)`	Split on a pattern
`re.compile(p)`	`Pattern` object with the same methods

A **`Match` object** has `.group()`, `.group(1)` / `.group("name")`, `.groups()`, `.groupdict()`, `.start()`, `.end()`, `.span()`.

Code

```
LOG = """2024-03-09 12:01:07 ERROR db timeout user=alice
2024-03-09 12:01:09 INFO  login ok user=bob
2024-03-10 08:15:00 ERROR disk full user=carol"""

s = "Order #123 shipped"
print(re.match(r"\d+", s))                  # None: match only tries at position 0
m = re.search(r"\d+", s)                    # scans the whole string
print(m, m.group(), m.start(), m.end(), m.span())
print(re.fullmatch(r"\d{3}", "123") is not None, re.fullmatch(r"\d{3}", "1234"))

print(re.findall(r"user=(\w+)", LOG))        # one group -> list of strings
print(re.findall(r"(\d{4})-(\d\d)-(\d\d)", "2024-03-09 and 2025-01-02"))  # tuples
for m in re.finditer(r"ERROR (.+?) user=(\w+)", LOG):
    print("  ", m.group(1), "->", m.group(2), "at", m.start())
```

Output:

```
None
<re.Match object; span=(7, 10), match='123'> 123 7 10 (7, 10)
True None
['alice', 'bob', 'carol']
[('2024', '03', '09'), ('2025', '01', '02')]
   db timeout -> alice at 20
   disk full -> carol at 112
```

```
pat = re.compile(r"(?P<date>\d{4}-\d\d-\d\d) (?P<time>[\d:]+) (?P<level>\w+)")
m = pat.search(LOG)
print(m.group("level"), m.group(1), m.groups())
print(m.groupdict())
print(re.search(r"(?:ab)+", "xxababab").group())   # (?:...) groups without capturing

print(re.sub(r"\s+", " ", "too    many   spaces"))
print(re.sub(r"(\d{4})-(\d\d)-(\d\d)", r"\3/\2/\1", "due 2024-03-09"))   # backrefs
print(re.sub(r"\d+", lambda m: str(int(m.group()) * 2), "3 apples, 10 pears"))
print(re.split(r"[,;]\s*", "a, b;c ,d"))

html = "<b>bold</b><i>it</i>"
print(re.findall(r"<.+>", html))     # greedy: grabs as much as possible
print(re.findall(r"<.+?>", html))    # lazy: as little as possible
print(re.escape("price: $5.00 (approx)"))
```

Output:

```
ERROR 2024-03-09 ('2024-03-09', '12:01:07', 'ERROR')
{'date': '2024-03-09', 'time': '12:01:07', 'level': 'ERROR'}
ababab
too many spaces
due 09/03/2024
6 apples, 20 pears
['a', 'b', 'c ', 'd']
['<b>bold</b><i>it</i>']
['<b>', '</b>', '<i>', '</i>']
price:\ \$5\.00\ \(approx\)
```

The date rewrite captures year, month, day as groups 1–3 and `r"\3/\2/\1"` writes them reversed (raw, so `\3` isn't an octal escape). `re.escape` escapes every special character — use it when inserting user text into a pattern.

Trade-offs

Task	Best tool
Fixed substring test	`in`, `startswith`, `endswith` — faster, clearer
Split on one fixed separator	`str.split`
Variable shapes, extraction, validation	`re`
HTML / XML	A real parser (`html.parser`, `lxml`, BeautifulSoup) — regex can't handle nesting
Strict email / URL validation	A dedicated library or `urllib.parse`

When not to use it

When a string method does the job, for nested formats (HTML, JSON, code), or with user-supplied patterns (see backtracking).

Common mistakes

- **Forgetting `r"..."`**: `"\b"` is a backspace.
- **`match` vs `search`**: `match` checks only the start; validate with `fullmatch`.
- **Greedy `.*`** swallowing too much — `.*?` or a negated class `[^<]*`.
- **Catastrophic backtracking:** Python's engine backtracks (retries alternatives when a path fails), so nested quantifiers (repeat counts like `*`, `+`, `{m,n}`) such as `(a+)+$` can take exponential time on near-matches, and simpler ones (`.*.*=.*`, `\s+$` on a long run of spaces) can go quadratic — the basis of ReDoS (regular-expression denial-of-service).
- `m.group()` without checking `m is not None` → `AttributeError`.

Real-world examples

- 2 July 2019: one new WAF (web application firewall) rule containing `.*(?:.*=.*)` backtracked so heavily it pinned CPUs at 100 % across Cloudflare's network — a global outage of about 27 minutes (Cloudflare post-mortem).
- 20 July 2016: Stack Overflow down 34 minutes — a trimming regex (`[\s\u200c]+$`) hit a post with ~20,000 consecutive whitespace characters and backtracked quadratically, O(n²), ~200 million steps for one string.
- Django routes URLs with regex via `re_path()` (before Django 2.0, `url()` regex routes were the only option); Logstash's grok filter is built on named regex patterns.

In the interview

> **30-second answer:** "`re.search` finds the first match anywhere, `match` only at the start, `fullmatch` the whole string. `findall` returns strings or tuples of groups, `finditer` gives Match objects, `sub` replaces with backrefs or a function. Use raw strings, lazy quantifiers when needed, named groups for readability, and beware catastrophic backtracking."

- *Why `re.compile`?* → Readability and reuse; `re` caches recent patterns, so speed gain is small.
- *`findall` with two groups?* → List of 2-tuples.
- *Non-capturing group?* → `(?:...)`.

Self-check

1. `re.match(r"\d", "a1")`? → `None`.
2. `re.sub(r"(\w+)@(\w+)", r"\2 at \1", "me@home")`? → `'home at me'`.

TL;DR

`search` / `match` / `fullmatch` for one match, `findall` / `finditer` for many, `sub` to replace, `()` to capture, `(?P<name>)` to name, `r""` always, keep patterns simple.

Chapter TL;DR

Topic	Remember
Basics	`str` is an immutable sequence of Unicode code points; zero-based and negative indexing.
Immutability	Methods and operators return new strings; rebind the name. Strings are hashable.
Operators	`+`, `*`, `in`; comparisons by code point; use `==`, not `is`.
Slicing	`[start:stop:step]`, stop excluded, never raises, `[::-1]` reverses.
Escapes	`\n \t \\ \' \" \xhh \uXXXX \N{...}`; `r"..."` for regex and paths.
Formatting	f-strings by default; `[[fill]align][sign][#][0][width][,][.prec][type]`; `format()` for stored templates, `%` for logging, `Template` for untrusted input.
Methods	`strip/split/join/partition/replace/startswith`; `in` not `find` for tests; `casefold` for comparisons; `isdecimal` for digits.
Building	Many pieces → list + `"".join`.
Unicode	Decode at input, encode at output, always `encoding="utf-8"`; normalise with NFC.
Regex	`search`, `match`, `fullmatch`, `findall`, `sub`, groups; raw strings; beware backtracking.